TOM KLOOTWIJK SIGNATURE EDITION

How a Tiny Substrate Becomes a 120 Hz Native Android App

An Explain Like I'm 5 technical report for UGTS-KC 3.9.1

The little-kid version: Tom's package is the recipe and the toy world. Android's standard tools are the factory. The Poco phone is the stage. The factory changes the recipe into phone-shaped machine code, but it does not invent the rules, scene, or visual identity.

Record	Value
Prepared for / requested author	Tom Klootwijk
Requester-supplied identifier	NL200678942
Requester-supplied date of birth	10-07-1990
Version examined	UGTS-KC 3.9.1 - Tom Klootwijk Signature Edition
Physical target	Poco X7 Pro 12 GB, model 2412DPC0AG
Measurement date	23 August 2026

Attribution and identity details are requester supplied. This technical report is not an identity check, legal ownership ruling, or cryptographic personal signature.

1. The answer in one minute

Bottom line: The deployed app is a small native C++ program whose game rules, scene, quality policy, and compact KC3D391 data come from the supplied 3.9.1 package. Existing Android tools compile and host it. No commercial game engine and no network service is involved.

Think of the project as five simple objects:

- The blueprint: project.json describes the arena, meshes, materials, rules, camera, light, and device tiers.
- The lunchbox: signature_scene.kc3d packs the blueprint into 21,735 bytes for fast native loading.
- The brain: 12 C++ source/header files implement the loop, input, movement, loading, device choice, adaptive quality, and drawing.
- The painter: OpenGL ES 3.0 asks the Mali-G720 GPU to draw colored triangles.
- The stage manager: Android NativeActivity supplies the window, lifecycle, touch/gamepad events, and access to the GPU driver.

Measured item	Result	Meaning
Presented frame rate	120.07 FPS mean	374 SurfaceFlinger presentation intervals
Frame interval	8.329 ms mean	8.528 ms p95; 8.770 ms maximum
App CPU	61.92% of one core	7.74% of total 8-core capacity
Memory	136.18 MiB PSS	251.48 MiB RSS; 64.49 MiB graphics
Thermal snapshot	Status 0 - no throttling	CPU/GPU 41.7 C; skin/battery 32.4 C

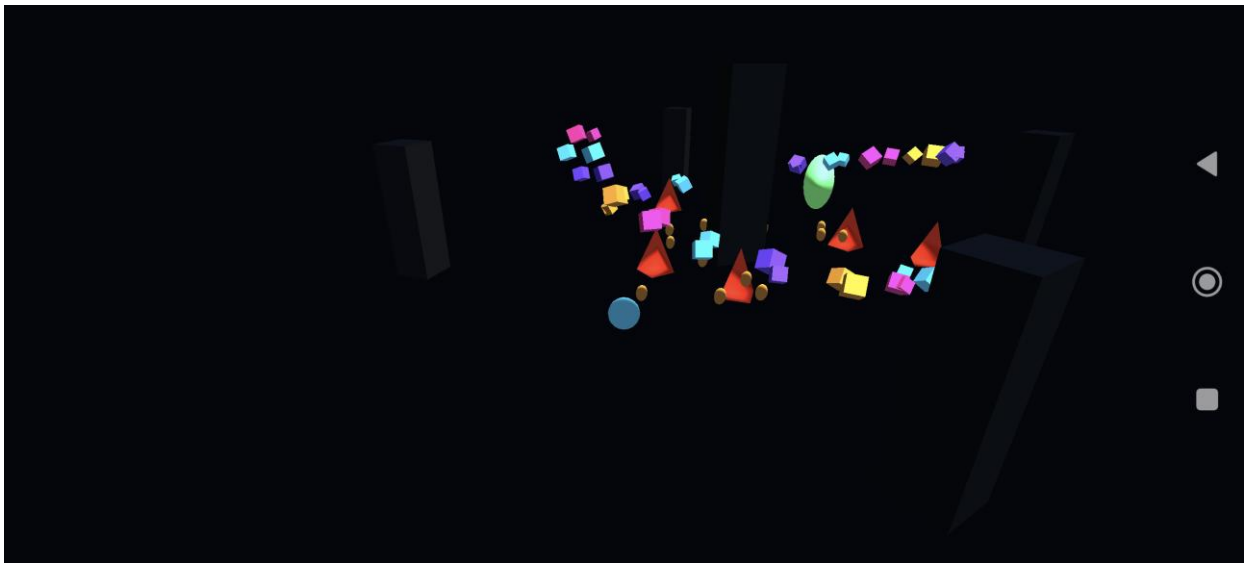

Figure 1. The actual deployed native scene captured from the Poco X7 Pro after the event-loop correction.

Important measurement boundary: These are short, foreground, USB-connected debug-build measurements with a static camera. They prove that this build presents smoothly at 120 Hz in this test. They are not a 30-minute heat, battery, gameplay, or release-build certification.

What this picture confirms

- The native activity reached a real EGL/OpenGL ES surface instead of remaining on a black window.
- The KC3D scene, shaders, camera, colors, depth, and shared meshes were loaded and drawn on the Poco.
- A screenshot proves visible output; the timing measurements later in this report are the separate performance evidence.

2. Where the deployed app came from

The source package already contains the authoring model, compiler, compact scene, Android template, checked-in Android project, validation records, and attribution files. The standalone project that was built is a copy of that Android tree.

A file-by-file lineage check

Twenty-seven build/runtime files were compared by SHA-256 between the supplied package and the deployed project:

- 26 files are byte-for-byte identical.
- 1 file differs: app/src/main/cpp/main.cpp.
- The difference is a minimal Android event-loop correction. It processes one looper result, returns to the outer loop, and lets the first frame render after focus changes.
- No scene, mesh, material, shader, gameplay rule, device profile, or art asset was added by that correction.

Why this honesty matters: The first archive version compiled and launched, but its stale infinite-wait timeout left the display black. The deployed APK therefore is not byte-for-byte the untouched archive. It is the archive plus this small platform-integration fix.

Integrity record	Value
Canonical project-content hash	e86bebe946cb6a4f7e418143869191c4900bed22d2773054e0b1bf0a6493c244
KC3D391 scene-pack SHA-256	485bb8a5ed7d4632577195dc0a889b4e6d3ea13ebc55482c5ea40761729ba437
Deployed debug APK SHA-256	e3229b25529d222dd1ff5d0f98ac5ddf2fb283d73b2a6b4208a4b6df054821cf
Application ID	nl.tomklootwijk.ugtskc.signature.pocox7pro
Version	391 / 3.9.1-poco-x7-pro

Authorship wording used in this report

This report describes the package as Tom Klootwijk's requested original/signature work because that is the attribution supplied by the requester and repeated in the package. The package itself says the identity, authorship, and rights assertion was not independently verified. The identifier NL200678942 and date 10-07-1990 are therefore presented as requester-supplied record data, not as a government verification.

3. From tiny description to running phone

Here is the whole factory line in child-friendly language, followed by the grown-up technical meaning.

1. **Write the toy-world recipe:** The editable project.json names every object, color, transform, rule, device tier, camera, and light. This is the source of truth.
2. **Fold the recipe into a lunchbox:** The package compiler converts the JSON into the little-endian KC3D391 binary scene pack. It stores validated counts, strings, meshes, materials, nodes, quality tiers, and target profiles.
3. **Read the factory card:** Gradle 8.13 and Android Gradle Plugin 8.13.2 read the application ID, SDK levels, ARM64 Poco flavor, assets, and native-build instructions.
4. **Build the metal brain:** CMake 3.22.1 and Ninja ask Android NDK r29's Clang compiler to turn six C++20 implementation files into libugts_kc_native.so for arm64-v8a.
5. **Pack the backpack:** AAPT2 and the Android packaging tasks combine the manifest, tiny resources, native library, KC3D scene, source JSON, inspection JSON, and two shader files into one APK.
6. **Put it on the phone:** ADB streams the APK to the Poco. Android verifies the debug signature and installs the application under its package name.
7. **Open the stage:** Android starts android.app.NativeActivity, creates a native window, and loads libugts_kc_native.so. No handwritten Java or Kotlin activity is required.
8. **Unpack the scene:** The C++ asset loader opens signature_scene.kc3d, validates its format, uploads four meshes to GPU buffers, and compiles the two GLSL ES shaders.
9. **Choose the Poco setting:** The explicit build flavor selects poco_x7_pro_12gb, requests 120 FPS, and starts with signature_ultra at full render scale.
10. **Play and paint:** The loop reads input, advances fixed-step gameplay, draws the scene into an offscreen framebuffer, copies it to the phone window, and swaps on vertical sync.

What Android tooling contributes: It contributes the compiler, packager, operating-system window, input plumbing, system libraries, graphics API, GPU driver, installer, and debug signature. It does not supply the scene design, KC3D data model, node rules, quality policy, or game-specific C++ logic.

4. What is actually used

Piece	When used	Actual job
Authoring source	Build-time	project.json is the editable record. It is compiled into KC3D391.
KC3D391 scene	Runtime	The engine reads 4 meshes, 8 materials, 66 nodes, 5 quality tiers, and 4 target profiles.
C++20 engine	Runtime	Lifecycle, input, fixed-step movement, collision checks, camera, quality logic, loading, and rendering.
NativeActivity + native_app_glue	Runtime/platform	Android lifecycle, native window, input queue, and looper integration.
EGL + OpenGL ES 3.0	Runtime/platform	Creates the graphics context, buffers, shaders, draws, blits, and presents frames.
Mali-G720 MC7 driver	Runtime/device	Executes the GLES commands on the Poco GPU.
JNI PowerManager call	Runtime/platform	Reads Android's current thermal status.
Android build toolchain	Build-time only	Gradle, AGP, JDK, CMake, Ninja, and NDK turn source into the ARM64 APK; they do not run as the game.

Things shipped but not opened by the current game loop

- assets/project.json is kept in the APK for inspection/provenance, but the C++ runtime opens signature_scene.kc3d instead.
- assets/scene-pack-inspection.json is also packaged for human/tool inspection, but the runtime does not read it.

Things in the wider package that are not running on this phone

- The Python reference oracle, schema validators, CLI, tests, 2D browser runtime, HTML5 exporter, glTF/USDA tools, reports, and diagrams.
- Unity, Unreal Engine, AndroidX, a web view, a Java game engine, cloud services, analytics, advertising, or network assets.
- Vulkan: it is only declared optional and documented as a future hook. The renderer is GLES3.
- 4D execution: it remains a design-only TODO and is not part of the running binary.
- Audio and textures: the current APK contains neither. The visual scene uses primitive meshes and flat/emissive material colors.
- Android permissions: the manifest requests none.

5. What happens inside one frame

1. **Listen:** Poll one Android looper result. Touch, keyboard, gamepad, focus, and window events enter here.
2. **Measure time:** Compute elapsed time and clamp it to 0.1 second so a long pause cannot create an enormous physics jump.
3. **Tick the rules:** Advance the world in exact 1/120-second slices: gravity, movement, floor/bounds, rotations, and player-vs-node contacts.
4. **Move the camera:** Orbit around the player target using yaw, pitch, and pinch distance.
5. **Check the thermostat:** About once each second, compare measured FPS and Android thermal status with the adaptive-quality thresholds.
6. **Paint offscreen:** Clear color/depth, set global light/camera uniforms, then issue one indexed draw for each visible live node.
7. **Copy and show:** Blit the offscreen color image to the Android window and call eglSwapBuffers with swap interval 1.

Scene workload

Shared mesh	Node draws	Triangles per draw	Triangles per frame
Cube	45	12	540
Floor	1	2	2
Pyramid	6	6	36
Sphere	14	440	6,160
Total	66	-	6,738

The four meshes contain only 306 unique vertex records and 460 unique triangles. Reusing them across 66 nodes produces the larger per-frame workload without duplicating the mesh buffers.

At 120 presented frames per second: 66 draws/frame becomes 7,920 draw calls/second; 6,738 triangles/frame becomes 808,560 triangles/second; 20,214 submitted indices/frame becomes 2,425,680 indices/second. This is a modest geometry load for a Mali-G720. The current renderer favors simplicity over batching: it does not use GPU instancing.

6. Pixel work and memory, calculated

The app is rotated to 2712 x 1220 landscape pixels. At full scale:

- Pixels per rendered image = 2712 x 1220 = 3,308,640 pixels (3.309 megapixels).
- Offscreen color = 4 bytes/pixel; depth is budgeted as 4 bytes/pixel for the DEPTH_COMPONENT24 renderbuffer allocation.
- Offscreen color + depth = 3,308,640 x 8 = 26,469,120 bytes = 25.24 MiB.
- The renderer clears the offscreen target and blits a full color image to the window: a minimum of two full-screen pixel passes before counting shaded geometry or system composition.
- At 120 FPS, that clear-plus-blit floor is about 794.1 million pixel operations per second.

Tier	Scale	Internal size	Mpix	% full	FBO MiB	2-pass Mpix/s at 120
Signature ultra	1.00	2712 x 1220	3.309	100.0%	25.24	794.1
High	0.92	2495 x 1122	2.799	84.6%	21.36	671.8
Balanced	0.82	2224 x 1000	2.224	67.2%	16.97	533.8
Safe	0.68	1844 x 830	1.531	46.3%	11.68	367.3
Thermal	0.55	1492 x 671	1.001	30.3%	7.64	240.3

The measured graphics allocation was 64.49 MiB, about 2.56 times the calculated offscreen color-plus-depth pair. The difference is reasonable because Android's window buffers, driver allocations, buffer queues, and other graphics bookkeeping are outside that simple two-buffer calculation.

A subtle but important fact: All five max-visible-node limits are above this scene's 66 nodes; even the thermal cap is 160. Therefore the current scene does not reduce draw count when quality steps down. In practice, the implemented fallback mainly reduces pixel resolution.

7. What the Poco actually measured

Test conditions: app in the foreground, static camera, screen awake, connected by USB/ADB, debug build, Android 16/API 36, landscape 2712 x 1220, active display mode 120 Hz.

Metric	Measured result	Method / meaning
Display mode	120.00001 Hz	Android display service; active mode ID 3
Presentation sample	374 intervals	Three cleared SurfaceFlinger windows
Mean presentation interval	8.329 ms	Equivalent to 120.07 presented FPS
Median / p95 / p99	8.324 / 8.528 / 8.640 ms	Tight pacing in the sampled static view
Maximum interval	8.770 ms	No interval exceeded 12.5 ms
Intervals above 1.5 vsync	0 of 374	0.00% in this short sample
Process CPU	61.92% of one core	Mean of three 3-second /proc samples
Whole-chip equivalent	7.74% of 8-core capacity	Simple 61.92 / 8 normalization
Total PSS	139,447 KiB = 136.18 MiB	Proportional memory snapshot
Total RSS	257,518 KiB = 251.48 MiB	Resident pages, including shared mappings
Graphics	66,042 KiB = 64.49 MiB	EGL + GL memory reported by dumpsys
Thermal status	0	Android says no thermal throttling state
HAL temperature snapshot	CPU/GPU 41.7 C	Battery/skin 32.4 C after restart/sample
Start after force-stop	235.3 ms mean	225-247 ms across three cached launches

How to read these numbers

- SurfaceFlinger measures when frames were presented. It is not a direct GPU render-duration trace.
- The 120.07 value is timing arithmetic around a nominal 120 Hz mode; report it as approximately 120 FPS, not as an overclock.
- CPU percent is process time. Some display/GPU work occurs in Android system or kernel processes and is not fully charged to the app process.
- RSS counts shared mappings more generously than PSS. PSS is the better single number for the app's proportional memory cost.
- The 235.3 ms result is ActivityManager's start-completion time after force-stop with warm file caches, not a camera-recorded first-interactive-pixel measurement.
- No battery-current, wattage, GPU counter, or long-duration thermal test was captured.

8. The quality thermostat - and its current limits

The quality controller is like a parent turning down a toy's detail when the phone gets tired.

- Stress is declared when Android thermal status is 3 or higher, or measured FPS is below 82% of 120 = 98.4 FPS.
- The configured stress timer is 1.5 seconds. Because the engine evaluates it only about once per second, a continuous problem normally needs two stressed checks before one tier is dropped.
- Recovery requires thermal status 1 or lower and FPS at or above 96% of 120 = 115.2 FPS for 8 seconds before moving up one tier.
- Only one tier changes per decision, preventing an immediate jump from ultra to thermal or back.

Index	Tier	Render scale	Node cap	Stored target FPS
0	Signature ultra	1.00	1024	120
1	High	0.92	720	90
2	Balanced	0.82	480	60
3	Safe	0.68	280	60
4	Thermal	0.55	160	45

Critical implementation facts

1. Stored target FPS values do not retune presentation: After startup, the app continues requesting 120 FPS and the adaptive comparison continues using the profile's 120 FPS target. The 90/60/45 values stored in lower tiers are parsed but are not used to change the frame-rate request.

2. Three quality fields are presently descriptive only: MSAA sample count, post-processing, and shadow quality are loaded from the scene pack, but the current GLES3 renderer does not implement them. There is no MSAA attachment, post-processing pass, or shadow pass.

3. Node caps do not affect this 66-node demo: Every tier permits at least 160 visible nodes. There is also no frustum culling or distance sorting; nodes are considered in stored order until the cap is reached.

These are not reasons to dismiss the design. They are the exact boundary between the compact policy encoded by the substrate and the subset that the present renderer actively enforces.

9. How compact is it?

Item	Footprint	Interpretation
Editable project JSON	160,201 bytes	Human-readable source record
KC3D391 binary pack	21,735 bytes	86.43% smaller; JSON is 7.37x larger
C++ implementation	12 files / 1,081 lines	42,874 bytes; 1,000 nonblank lines
GLSL shader source	1,093 bytes	One vertex shader + one fragment shader
Debug APK	1,173,398 bytes	1.119 MiB total archive
ARM64 native library	1,121,752 bytes	Uncompressed in APK; about 95.6% of APK size
Tiny DEX	1,396 bytes	No handwritten Java/Kotlin game code

Why the native library is the big piece

The project requests the static C++ standard library (c++_static). That library support is folded into libugts_kc_native.so instead of being a separate runtime dependency. The stripped binary's text section is about 1.081 MB, so the native code/support dominates the APK while the custom scene and shader data remain tiny.

Tiny substrate does not mean zero platform

The APK is small because it reuses facilities already present on Android: the operating system, Linux process model, NativeActivity framework, EGL, OpenGL ES API, Mali driver, display compositor, input system, power service, and installer. Those platform components are substantial, but they are not copied into the APK and they do not replace the package's game-specific design.

Most accurate one-sentence description: A compact Tom Klootwijk-attributed data/engine package is cross-compiled against standard Android native interfaces, then the Poco's existing OS and GPU execute that package directly.

10. What this proves, and what it does not

Demonstrated in this session

- The Poco-specific ARM64 debug APK compiles with SDK 36, NDK r29, CMake 3.22.1, and AGP/Gradle 8.13.x.
- It installs, launches, stays in the foreground, loads the Mali-G720 GLES3 driver, selects the 12 GB Poco profile, and visibly renders the scene.
- The active Android display mode is 120 Hz and the sampled presented-frame cadence is approximately 120 FPS with tight short-window pacing.
- Short CPU, memory, launch, and thermal snapshots were captured and are reported with their methods.
- The deployed source lineage matches the supplied Android project except for the disclosed event-loop correction.

Not demonstrated

- Sustained 120 FPS during 30-60 minutes of active gameplay, charging, high ambient temperature, or low-battery conditions.
- Battery drain, watts, GPU utilization/counters, per-pass GPU duration, or energy efficiency.
- Release-mode performance, release signing, Play Store readiness, security review, or production certification.
- Robust general physics, skeletal animation, networking, multiplayer, anti-cheat, Vulkan rendering, or a 4D runtime.
- Independent verification of identity, authorship, ownership, or third-party rights.

Most useful next benchmark

For a production performance claim, the next honest test is a 30-minute release-build run with automated movement/camera input, Perfetto CPU/GPU counters where available, frame timeline data, battery current, and temperature samples every minute. That would test sustained behavior instead of the lightweight static view measured here.

11. Evidence record and formulas

Primary package records

Package root:
`C:\Users\Tom\Documents\theTomKlootwijkManifold\UGTS_KC_Tom_Klootwijk_Signature_3_9_1_Package\UGTS_KC_Tom_Klootwijk_Signature_3_9_1_Package`

Signature record: `signature\TOM_KLOOTWIJK_SIGNATURE_EDITION.json`

Native contract: `spec\MOBILE_3D_AND_ANDROID_CONTRACT.md`

Scene-pack format: `spec\NATIVE_SCENE_PACK_3_9_1.md`

Evidence boundary: `docs\EVIDENCE_BOUNDARY.md`

Android guide: `docs\ANDROID_NATIVE_GUIDE.md`

Checked-in project: `android\UGTSKC391Signature`

Editable scene: `examples\tom_signature_arena_3d\project.json`

Deployed implementation records

Built project:
`C:\Users\Tom\Documents\theTomKlootwijkManifold\UGTS_KC_3_9_1_Tom_Signature_Android_Source\UGTSKC391Signature`

Event-loop correction: `app\src\main\cpp\main.cpp`

Runtime engine: `app\src\main\cpp\engine.cpp`

GLES3 renderer: `app\src\main\cpp\renderer_gles3.cpp`

Compact scene: `app\src\main\assets\signature_scene.kc3d`

Calculation formulas

- Triangles/frame = sum(node count using mesh x mesh index count / 3) = 6,738.
- Presented FPS = 1000 / mean presentation interval in milliseconds = 1000 / 8.329 = 120.07.
- Full pixels/frame = 2712 x 1220 = 3,308,640.
- Offscreen color+depth MiB = width x height x (4+4) / 1,048,576 = 25.24 MiB.
- Minimum two-pass pixel rate = pixels x 2 x 120 = 794,073,600 pixel operations/second.
- Process CPU = process CPU seconds / wall seconds x 100; whole-chip normalization = one-core percent / 8.
- PSS MiB = dumpsys KiB / 1024; 139,447 / 1024 = 136.18 MiB.
- Pack reduction = 1 - 21,735 / 160,201 = 86.43%.

Measurement interfaces

Android SurfaceFlinger --latency supplied presentation timestamps; dumpsys display supplied the active refresh mode; /proc/<pid>/stat supplied process CPU time; dumpsys meminfo supplied PSS/RSS/graphics memory; dumpsys thermalservice supplied status and temperatures; am start -W supplied ActivityManager start timing.

Small glossary

- Substrate: the compact rules, records, formats, and source that define the work.
- APK: the installable Android package placed on the phone.
- Native: machine code compiled for the phone's ARM64 processor instead of game logic running in JavaScript or a managed game engine.
- Frame: one complete picture presented to the display.
- PSS: the app's private memory plus its fair share of shared memory.
- Vsync: the display's regular heartbeat; at 120 Hz one beat is about 8.333 ms.

Final ELI5 summary: Tom's compact files tell the phone what world exists and how it behaves. Standard Android tools translate those files into the phone's language. Android opens a window, the C++ brain advances the toy world 120 times a second, and the Mali GPU colors the triangles. In the short test, the Poco kept pace with the 120 Hz screen without a sampled missed-vsync interval or thermal warning.